# Practical 8

In [2]:

```python
import gymnasium as gym
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np

# --------------------------------
# Hyperparameters
# --------------------------------
GAMMA = 0.99
LR = 0.005
EPISODES = 500

# --------------------------------
# Environment
# --------------------------------
env = gym.make("CartPole-v1")
state_dim = env.observation_space.shape[0]
action_dim = env.action_space.n

# --------------------------------
# Policy Network
# --------------------------------
class PolicyNetwork(nn.Module):
    def __init__(self, state_dim, action_dim):
        super().__init__()
        self.fc1 = nn.Linear(state_dim, 128)
        self.fc2 = nn.Linear(128, action_dim)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        return torch.softmax(self.fc2(x), dim=-1)

policy = PolicyNetwork(state_dim, action_dim)
optimizer = optim.Adam(policy.parameters(), lr=LR)

# --------------------------------
# Select Action
# --------------------------------
def select_action(state):
    state = torch.tensor(state, dtype=torch.float32)
    probs = policy(state)
    dist = torch.distributions.Categorical(probs)
    action = dist.sample()
    return action.item(), dist.log_prob(action)

# --------------------------------
# Compute Returns
# --------------------------------
def compute_returns(rewards):
    returns = []
    G = 0
    for r in reversed(rewards):
        G = r + GAMMA * G
        returns.insert(0, G)
    return torch.tensor(returns, dtype=torch.float32)

# --------------------------------
# Training Loop (Improved REINFORCE)
# --------------------------------
reward_history = []

for episode in range(EPISODES):
    state, _ = env.reset()

    log_probs = []
```

```
    rewards = []

    while True:
        action, log_prob = select_action(state)

        next_state, reward, terminated, truncated, _ = env.step(action)
        done = terminated or truncated

        log_probs.append(log_prob)
        rewards.append(reward)

        state = next_state

        if done:
            break

    # Compute returns
    returns = compute_returns(rewards)

    #  Baseline + Advantage
    baseline = returns.mean()
    advantages = returns - baseline

    # Normalize advantages
    advantages = (advantages - advantages.mean()) / (advantages.std() + 1e-9)

    #  Vectorized Loss
    log_probs = torch.stack(log_probs)
    loss = -torch.sum(log_probs * advantages)

    # Update policy
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    # Track rewards
    total_reward = sum(rewards)
    reward_history.append(total_reward)

    # Print smoother progress
    if episode % 50 == 0:
        avg_reward = np.mean(reward_history[-50:])
        print(f"Episode {episode}, Avg Reward: {avg_reward:.2f}")

env.close()
```

```
Episode 0, Avg Reward: 29.00
Episode 50, Avg Reward: 24.50
Episode 100, Avg Reward: 85.98
Episode 150, Avg Reward: 78.62
Episode 200, Avg Reward: 128.80
Episode 250, Avg Reward: 126.04
Episode 300, Avg Reward: 182.62
Episode 350, Avg Reward: 257.04
Episode 400, Avg Reward: 387.70
Episode 450, Avg Reward: 496.34
```